

Guide 3: Setting Up and Using a GitHub Repository

This guide walks through creating your own GitHub repository from nothing, getting it set up on your machine, and the everyday workflow for adding and changing files - plus what to do when things go wrong.

GitHub Uses Version Control

Version control is like a detailed "save history" for your files. Every time you save a change (a "commit"), git remembers what changed, when, and you can always look back at - or even restore - any previous version.

Think of it like a video game's save system: instead of one save file that gets overwritten, you have a whole timeline of saves you can jump back to if something goes wrong, or just to see how things looked before.

GitHub is built around this idea - it hosts your repository online, keeps the full history of every change, and lets you (or others) view, compare, and restore previous versions at any time. For a project with multiple files (like a folder of documents, code, or notes), this means you never lose work, can track how things evolved over time, and can experiment freely - knowing you can always go back.

Step 1 - Setting Up a Repository from Scratch

1. Create a New Repository on GitHub

- Go to github.com and click "New repository" (usually via a "+" icon top-right)
- Repository name - choose something descriptive (lowercase, hyphens instead of spaces is conventional, e.g. my-project)
- Description - optional, but useful for explaining what the repo is for
- Visibility - "Public" (anyone can view) or "Private" (only you/invited collaborators)
- Add a README file - tick this, as it gives you an initial file to work with
- Add .gitignore - optional, can be added later
- Choose a license - if your repo is public and you want others to know how they can use your code, a license like MIT is a common permissive choice
- Click "Create repository"

What kinds of repos can you make?

While GitHub is most commonly associated with code/tech projects, a git repo is really just a folder with version-controlled files - it can contain anything: text, markdown, images, PDFs, spreadsheets, etc.

People use GitHub for all sorts of non-code projects: gardening logs, recipe collections, personal journals or wikis, budget spreadsheets, photography portfolios, and more. The version history, free hosting, and markdown rendering make it a genuinely useful tool beyond just software.

That said, GitHub's audience and culture are tech-leaning, so non-tech repos are less common - but there's nothing stopping you, and it's a great way to learn git/GitHub skills that transfer to any project.

2. Make Managing Your Repo Easier - Clone It

What you've just created lives on GitHub's servers - this is your remote repository. You can edit files directly on github.com through the browser, but this is clunky for anything beyond small text edits.

Cloning creates a local copy of the repo on your own machine (e.g. your server). This lets you:

- Edit files with familiar tools (vim, your terminal)
- Work offline, then push changes when ready
- Use git commands to track changes, history, and sync with GitHub

Once cloned, you have two copies: your local repo (on your machine) and the remote repo (on GitHub). The git pull and git push commands keep these two in sync - pull brings remote changes to your local copy, push sends local changes to the remote.

3. Clone the Repository Locally

These commands work the same way across Linux terminal, Windows PowerShell/Command Prompt, and Mac Terminal - git commands are consistent across platforms, as long as git is installed (Windows users may need to install Git for Windows first).

Navigate to where you want the repo to live, then:

```
git clone https://github.com/yourusername/your-repo-name.git
```

This creates a new folder named after your repo, containing a local copy of everything.

```
cd your-repo-name
```

4. Configure Git (First Time Only)

Git needs to know who you are, for commit history:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Use the same email as your GitHub account if you want commits linked to your profile.

5. Set Up Credential Storage

By default, git asks for your GitHub username/password (or token) on every push. To avoid this:

```
git config --global credential.helper store
```

The first time you push, you'll be prompted - enter your GitHub username, and for the password, use a Personal Access Token (not your actual GitHub password - GitHub no longer accepts those for git operations). You can generate a token under GitHub Settings → Developer settings → Personal access tokens. After the first successful push, credentials are saved and you won't be asked again.

Step 2 - The Core Workflow: Add, Commit, Push

This is the everyday cycle you'll use whenever you change anything in your repo.

1. Edit a File with Vim

```
vim README.md
```

Press i to enter insert mode and start typing/editing. When finished, press Esc to exit insert mode, then type :wq and press Enter to save and quit.

2. Check What's Changed

```
git status
```

This shows which files have been modified, added, or deleted since your last commit - useful for double-checking before you proceed.

3. Stage Your Changes - git add

```
git add README.md
```

This tells git "include this file's changes in the next commit." Think of it as putting items into a box before sealing it. You can add multiple files individually, or use `git add .` to stage everything that's changed in the current directory and subdirectories.

4. Commit Your Changes

```
git commit -m "Brief description of what changed"
```

This "seals the box" - creating a permanent snapshot in your repo's history, labeled with your message.

Writing good commit messages: Keep them short but descriptive - someone (including future you) should be able to understand what changed just from the message. Good: "Add installation steps to README". Less useful: "updates" or "fix".

5. Push to GitHub

```
git push
```

This sends your committed changes from your local repo to the remote repo on GitHub - making them visible online and keeping both copies in sync.

The Full Cycle, Summarised

```
vim filename.md      # edit
git add filename.md  # stage
git commit -m "..." # commit
git push             # sync to GitHub
```

This four-step cycle is something you'll repeat constantly - it becomes second nature quickly.

Step 3 - Adding Directories and Files

These commands work the same across Linux terminal, Windows PowerShell/Command Prompt, and Mac Terminal.

1. Creating a New Directory (Folder)

```
mkdir new-folder-name
```

This creates a new folder inside your current location. Use `cd new-folder-name` to move into it, or stay where you are and reference the folder path directly when creating files.

2. Why Git Doesn't Track Empty Folders

Git tracks files, not folders directly - an empty folder has nothing in it to track, so git effectively ignores it. If you create an empty `images/` folder and try to `git add` it, nothing happens.

This isn't usually a problem - as soon as you add a file inside the folder, git starts tracking the folder automatically as part of that file's path.

3. Creating a New File

Using vim, as covered in Step 2:

```
vim new-folder-name/new-file.md
```

Vim will create the file (and, if needed, the path leading to it - though not the folders themselves, see note below) when you save with :wq.

Note: Vim can create a file inside a folder that already exists, but won't create the folder itself if it doesn't exist - you'll get a "Cannot open file for writing" error. Always mkdir the folder first (step 1), then create files inside it.

4. Adding and Pushing New Files/Folders

Once your file is created and saved:

```
git add new-folder-name/new-file.md
git commit -m "Add new file describing X"
git push
```

Or, to add everything new/changed in one go:

```
git add .
git commit -m "Add new folder and file"
git push
```

Summary

To add a new section to your repo: create the folder with mkdir, create your file inside it with vim (saving will populate the folder), then git add, commit, and push as usual. The folder itself doesn't need any special treatment - it simply "appears" in your repo once it contains a tracked file.

Step 4 - Adding Images and PDFs

1. Embedding Images in Markdown

To display an image directly within a markdown file (like a README), use:

```
![Description of image](path/to/image.jpg)
```

The ! makes it an embedded image rather than a clickable link - it renders directly on the page. The text in square brackets is alt text (shown if the image fails to load, and read aloud by screen readers).

2. Linking to PDFs and Other Files

PDFs can't be embedded inline the same way - instead, link to them so readers can click to open/download:

```
[Description of the document](path/to/document.pdf)
```

Without the !, this becomes a regular clickable link rather than an embedded image - clicking it opens the PDF in a new tab (or downloads it, depending on the browser).

3. Getting Files Into Your Repo

There are two main approaches:

Option A - Upload via GitHub's website

Navigate to the folder where you want the file, click "Add file" → "Upload files", then drag in your image/PDF and commit. This is the simplest option when the file is already on the computer or phone you're currently using to browse GitHub.

Option B - Copy to your server, then add via git

If the file is on a different device (e.g. a photo on your phone, or a document created on your Windows PC) and your repo lives on your server, you'll need to transfer the file across first. A few options:

- WinSCP - a free Windows tool for transferring files to/from a Linux server over SSH, using a simple drag-and-drop file browser interface
- Cloud storage (Google Drive, Dropbox, etc.) - upload the file from one device, then download it on the other
- USB transfer - copy the file onto a USB stick, then plug it into the target machine

Once the file is on the same machine as your repo, use the standard git add/commit/push cycle from Step 2.

4. Renaming or Moving Files - git mv

If a file needs renaming, or moving into a different folder, use:

```
git mv old-path/old-name.ext new-path/new-name.ext
```

This single command renames/moves the file and prepares it to be committed - so unlike a normal edit, you don't need a separate git add step here. Afterwards, continue as normal:

```
git commit -m "Rename/move file"
git push
```

Note: Renaming via GitHub's website upload interface can be unreliable (sometimes creating a file with a literal slash in its name, rather than a new folder). git mv locally is the more dependable method.

Step 5 - What Can Go Wrong (and How to Fix It)

This step covers common error messages you'll encounter, why they happen, and how to resolve them.

Problem 1: "Rejected - fetch first"

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs...
hint: Updates were rejected because the remote contains work that you
do not have locally.
```

Why this happens: Your local repo and the GitHub (remote) repo have both changed independently - perhaps you edited a file via GitHub's website, while also making local commits. Git won't let you push, because doing so could overwrite those remote changes.

The fix:

```
git pull
```

This downloads the remote changes and attempts to combine them with your local changes. If the changes are to different files (or different parts of the same file), this usually completes automatically. Then:

```
git push
```

Problem 2: "Divergent branches"

hint: You have divergent branches and need to specify how to reconcile them.

fatal: Need to specify how to reconcile divergent branches.

Why this happens: This often appears right after the "rejected" error above, during git pull. Git found that your local and remote histories have diverged (both have commits the other doesn't have), and needs to know how you want to combine them - there's more than one valid approach.

The fix:

```
git config pull.rebase false
git pull
```

pull.rebase false tells git to use merge - the simplest strategy, which combines both histories and creates a "merge commit" recording that combination. This is a one-time setting (saved for future pulls in this repo). After running this, git pull may open vim with a pre-filled merge commit message - just save and quit (:wq) to accept it.

Problem 3: "Bad source" / Files Not Found

fatal: bad source, source=..., destination=...

or git add silently does nothing for a file you expect to see.

Why this happens: Usually one of:

- Wrong directory - you're not inside the repo folder (or not in the right subfolder). Use pwd to check where you are (works in Linux, Mac, and modern Windows PowerShell), and ls (or dir on older Windows Command Prompt) to see what's there
- Local repo is out of date - the file exists on GitHub (e.g. uploaded via the website) but you haven't git pull-ed yet, so your local copy doesn't have it
- Typo or case-sensitivity - GitHub's filesystem is case-sensitive (README.md vs readme.md are different files), even if your local OS isn't

The fix: Check you're in the right folder (pwd/ls), run git pull to sync any remote changes, and double-check spelling/capitalisation matches exactly.

Problem 4: Markdown Renders as a Code Block Instead of a List

You write a nested bullet point, but on GitHub it displays in a grey monospace box instead of as a normal list item.

Why this happens: In markdown, any line indented by 4 or more spaces is treated as a "code block" - this is a long-standing markdown convention, originally meant for displaying code samples with preserved formatting.

The fix: For nested list items, use only 2 spaces of indentation, not 4 or a full tab:

```
- Main item
  - Nested item (2 spaces - renders as a list)
```

```
- Main item
  - Nested item (4 spaces - renders as a code block!)
```

If you spot a section rendering as a grey box on GitHub when it shouldn't be, check the indentation level first.

Guide 3: Setting Up and Using a GitHub Repository - Summary

Goal: Create a GitHub repository from scratch, set it up for local editing, and learn the everyday workflow for adding, changing, and troubleshooting files.

What this guide covers:

- GitHub Uses Version Control - A simple explanation of what version control is, and why it's useful for any kind of project, not just code.
- Setting Up a Repository - Creating a new repo on GitHub, understanding why cloning it locally is worthwhile, cloning it to your machine, and configuring git (username, email, credential storage) for the first time.
- The Core Workflow - The everyday cycle: edit with vim, check status, git add, git commit, git push. Includes tips on writing useful commit messages.
- Adding Directories and Files - Creating new folders, why git doesn't track empty ones, and adding new files within them.
- Adding Images and PDFs - Embedding images vs linking to documents in markdown, getting files onto your repo from different devices, and renaming/moving files with git mv.
- Troubleshooting - Four common problems explained in depth: "rejected - fetch first", divergent branches, "bad source"/files not found, and markdown rendering as a code block instead of a list.

Key takeaway: The add/commit/push cycle is the heart of working with git - everything else (branches, folders, images, troubleshooting) builds on that same foundation. Most errors come down to two things: your local and remote copies being out of sync (git pull fixes this), or a small formatting/path detail (checking pwd, spelling, and indentation fixes this).